

Développement mobile

Software Development Kit (SDK) Flutter

Langage DART

Dr Ibrahima GAYE
ibrahima2.gaye@uadb.edu.sn

Universite Alioune DIOP (UAD) de Bambey

5 novembre 2021



Références

- ▶ <https://flutter.dev/docs/get-started/install/windows>
- ▶ <https://www.udemy.com/course/flutter-dart-creez-des-applications-pour-ios-et-android/>
- ▶ <https://www.youtube.com/watch?v=LgbB3e7xuiA>
- ▶ <https://flutter-love.com/introduction-a-flutter/>

Langage de programmation Dart, les fondamentaux

Syntaxe de base

Fonctions

Programmation Orientée Objet

Dart

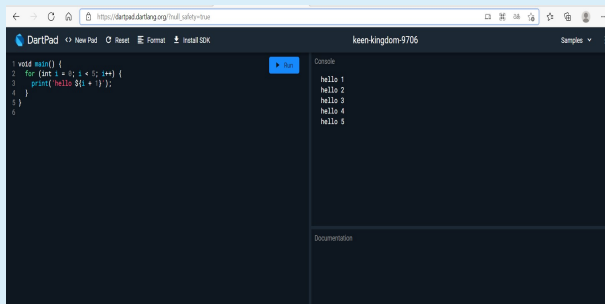
- ▶ Un langage de programmation Objet et Généraliste
- ▶ Développé par Google ; la version 1.0 a vu le jour en 2011
- ▶ Approuvé par la suite par Ecma (ECMA-408) en 2014.
- ▶ Objectif : remplacer JavaScript mais n'a jamais réussi
- ▶ Choisi comme langage du SDK Flutter, une nouvelle version a vu le jour en 2018
 - ▶ Rapide
 - ▶ intuitive
 - ▶ Simple

Dart

- ▶ On peut faire applications :
 - ▶ Web
 - ▶ Serveur
 - ▶ Mobiles
 - ▶ Pour les IoT
- ▶ Syntaxe très proche de celle du langage C
- ▶ Typage statique et un système de type sonore.

Interpréteur DART

- ▶ Existence d'un outil : DARTPAD
- ▶ <https://dartpad.dartlang.org/>



Langage de programmation Dart, les fondamentaux

Syntaxe de base

Fonctions

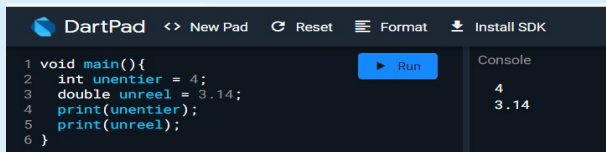
Programmation Orientée Objet

Variables

- ▶ DART supporte les types :
 - ▶ Les nombres (entiers : int et les réels : double)
 - ▶ Les chaînes de caractères (String)
 - ▶ Les booléennes (bool)
 - ▶ Les types dynamique (var)

Variables

► Les nombres



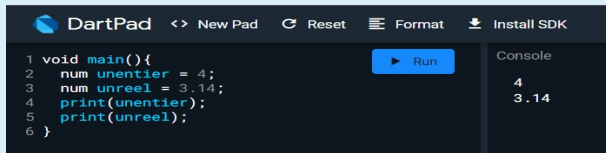
```
1 void main(){
2   int unentier = 4;
3   double unreel = 3.14;
4   print(unentier);
5   print(unreel);
6 }
```

The screenshot shows the DartPad interface. The code editor contains a Dart program that declares an integer variable `unentier` with the value 4 and a double variable `unreel` with the value 3.14. Both variables are printed to the console. The console output shows the values 4 and 3.14 on separate lines. The interface includes a 'Run' button and a 'Console' panel.

- un nombre non initialisé a une valeur **null**
 - DART les voit comme des objets

Variables

- ▶ int et double hérite de la classe **num**



The screenshot shows the DartPad web IDE interface. The top bar includes the DartPad logo and buttons for 'New Pad', 'Reset', 'Format', and 'Install SDK'. The main editor area contains the following Dart code:

```
1 void main(){
2   num unentier = 4;
3   num unreel = 3.14;
4   print(unentier);
5   print(unreel);
6 }
```

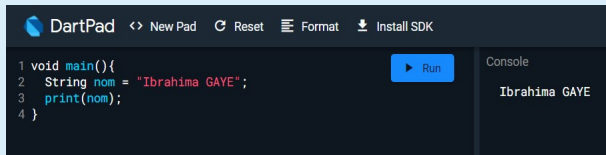
To the right of the code is a 'Run' button. Below the code editor is a 'Console' panel displaying the output of the program:

```
4
3.14
```

- ▶ Possible de convertir une chaîne numérique avec la syntaxe : **int.tryParse("32")** ;

Variables

- ▶ Les chaînes de caractères : **String**
 - ▶ une valeur entourée par des double quotes (guillemet) ou des quotes simple (apostrophe)
- ▶ Exemple :



The screenshot shows the DartPad web interface. At the top, there is a toolbar with icons for 'New Pad', 'Reset', 'Format', and 'Install SDK'. Below the toolbar, the code editor contains the following Dart code:

```
1 void main(){  
2   String nom = "Ibrahima GAYE";  
3   print(nom);  
4 }
```

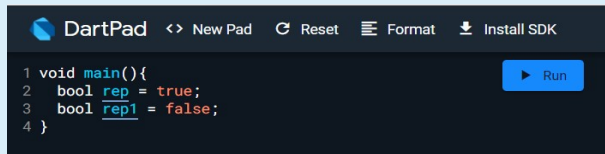
To the right of the code editor, there is a 'Run' button. Below the code editor, there is a 'Console' panel that displays the output of the program: 'Ibrahima GAYE'.

Variables

- ▶ Opérations sur les chaînes
 - ▶ Comme dans les autres langages, DART offre la possibilité de manipuler les chaînes de caractère
 - ▶ la fonction `toString` : convertit un nombre en une chaîne (Syntaxe : `unnombre.toString()`)
 - ▶ Conversion en Majuscule, Minuscule : `toLowerCase()` et `toUpperCase()`
 - ▶ fonction d'affichage : `print(chaine)`
 - ▶ Extraire une sous chaîne dans une chaîne : `substring(indiceDebut, indiceFin)`
 - ▶ etc.

Variables

- ▶ Les booléens
 - ▶ le type **bool** avec ses deux valeurs **true** et **false**



The image shows a screenshot of the DartPad web editor. The interface has a dark theme. At the top, there is a toolbar with icons for 'New Pad', 'Reset', 'Format', and 'Install SDK'. Below the toolbar, the code editor contains the following Dart code:

```
1 void main(){
2   bool rep = true;
3   bool rep1 = false;
4 }
```

To the right of the code editor, there is a blue button with a play icon and the text 'Run'.

Variables

- ▶ Type dynamique
 - ▶ Dès fois, on ne souhaite pas typer une variable lors de sa création
 - ▶ le type nous permet de créer une variable et mettre des valeurs de natures différentes
- ▶ DART propose deux types :
 - ▶ **var** : la variable aura comme type celui de la premier valeur qu'on lui a affecté
 - ▶ **dynamic** : la variable peut recevoir toutes les valeurs

Variables

- ▶ Type dynamique

- ▶ var :

```
1 void main(){
2   var unevariable;
3
4   unevariable = 'DIOP';
5   print(unevariable);
6
7   unevariable = 14;
8   print(unevariable);
9
10 }
```

Run

Console

DIOP
14

info

line 2 • Prefer typing uninitialized variables and fields. [\(view docs\)](#)

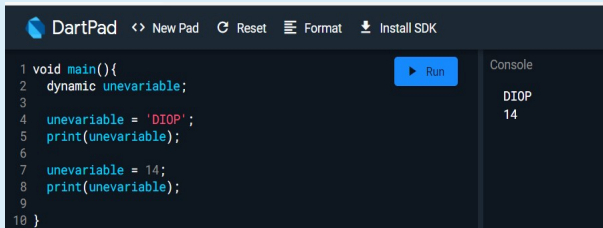
1 issue [hide](#)

Based on Flutter 2.5.3 Dart SDK 2.14.4



Variables

- ▶ Type dynamique
 - ▶ dynamic



```
1 void main(){
2   dynamic unevariable;
3
4   unevariable = 'DIOP';
5   print(unevariable);
6
7   unevariable = 14;
8   print(unevariable);
9
10 }
```

Console

DIOP
14

Les constantes

- ▶ DART propose deux mots clés pour définir une constante
 - ▶ **final** : l'objet ne peut être modifié ; une tentative de modification ne génère pas exceptions ; l'opération sera ignorée.
 - ▶ **const** : l'objet ne peut être modifié ; une tentative de modification génère une exception

Les collections

- ▶ DART propose plusieurs collections et les deux les plus utilisées sont :
 - ▶ Les Listes : une collection où chaque item est associé à un index
 - ▶ les Maps : une collection qui fonctionne sur un ensemble clé/valeur

Les collections

► Les listes

- Une collection où chaque valeur associée à un index
- Le premier index est 0
- La taille est fixe (Comme un tableau) ou variable (Comme une liste chaînée)
- Syntaxe de déclaration : `List<int> listenombre = new List(10);`
 - une liste de 10 int
 - les fonctions comme `add`, etc. ne peuvent pas être utilisées
- Plus de flexibilité : `List<int> listenombre = List<int>();`
 - noter l'absence du mot-clé `new` (Pas obligatoire dans les dernières versions)

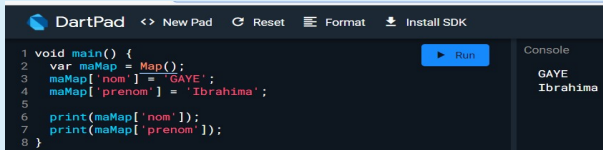
Les collections

- ▶ Comme sous JAVA, on peut utiliser les fonctions dans les collections
 - ▶ `List.add(el)` : Ajouter un élément dans la liste
 - ▶ `List.clear()` : vider la liste
 - ▶ `List.remove(el)` : Supprimer l'élément `el` dans la liste ; si c'est présent en plusieurs occurrences, la première sera supprimée
 - ▶ `List.length()` : Renvoyer la taille de la liste

Les collections

► Les maps

- Une autre catégorie de collections que fonctionnent sur un système **CLE/VALEUR**
- Syntaxe et utilisation :



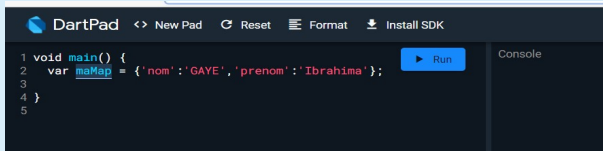
The screenshot shows the DartPad interface with a code editor and a console. The code defines a map and prints its values.

```
1 void main() {  
2   var maMap = Map();  
3   maMap['nom'] = 'GAYE';  
4   maMap['prenom'] = 'Ibrahima';  
5  
6   print(maMap['nom']);  
7   print(maMap['prenom']);  
8 }  
9
```

The console on the right displays the output:

```
GAYE  
Ibrahima
```

- Possible d'initialiser directement le map



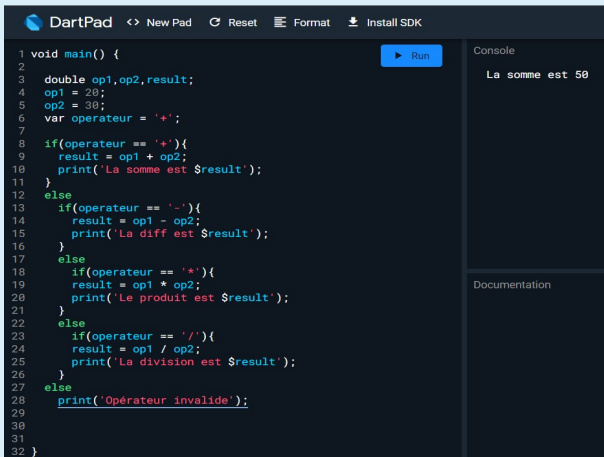
The screenshot shows the DartPad interface with a code editor and a console. The code initializes a map directly.

```
1 void main() {  
2   var maMap = {'nom': 'GAYE', 'prenom': 'Ibrahima'};  
3  
4 }  
5
```

Syntaxe de base

Structure alternative

- ▶ Même syntaxe en programmation C, JAVA, etc.

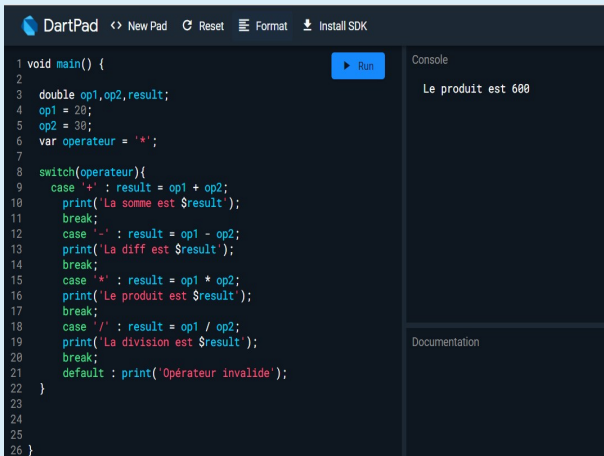


The screenshot shows the DartPad web editor interface. The top bar includes the DartPad logo and buttons for 'New Pad', 'Reset', 'Format', and 'Install SDK'. The main editor area contains a Dart program that implements a simple calculator using a switch statement. The program defines a `main` function with two variables, `op1` and `op2`, and a variable `opérateur` for the operator. It uses `if-else` blocks to handle addition, subtraction, multiplication, and division, printing the result for each case. A default case prints an error message for an invalid operator. A 'Run' button is located to the right of the code editor. On the right side of the interface, there is a 'Console' panel showing the output 'La somme est 50' and a 'Documentation' panel.

```
1 void main() {  
2  
3   double op1,op2,result;  
4   op1 = 20;  
5   op2 = 30;  
6   var opérateur = '+';  
7  
8   if(opérateur == '+'){  
9     result = op1 + op2;  
10    print('La somme est $result');  
11  }  
12  else  
13    if(opérateur == '-'){  
14      result = op1 - op2;  
15      print('La diff est $result');  
16    }  
17    else  
18      if(opérateur == '*'){  
19        result = op1 * op2;  
20        print('Le produit est $result');  
21      }  
22      else  
23        if(opérateur == '/'){  
24          result = op1 / op2;  
25          print('La division est $result');  
26        }  
27      else  
28        print('Opérateur invalide');  
29  
30  
31  
32 }
```

Structure alternative

- La structure switch peut être utilisée



```
1 void main() {  
2  
3   double op1,op2,result;  
4   op1 = 20;  
5   op2 = 30;  
6   var operateur = '*';  
7  
8   switch(operateur){  
9     case '+': result = op1 + op2;  
10    print('La somme est $result');  
11    break;  
12    case '-': result = op1 - op2;  
13    print('La diff est $result');  
14    break;  
15    case '*': result = op1 * op2;  
16    print('Le produit est $result');  
17    break;  
18    case '/': result = op1 / op2;  
19    print('La division est $result');  
20    break;  
21    default : print('Opérateur invalide');  
22  }  
23  
24  
25  
26 }
```

Run

Console

Le produit est 600

Documentation

Autres opérations

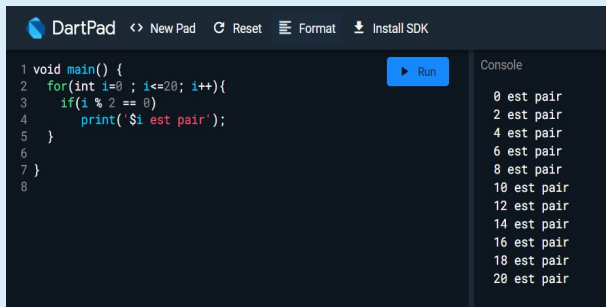
- ▶ Opération ternaire : `expressionBOOL ? Instruction1 : Instruction2 ;`
 - ▶ Si `expressionBOOL` est vraie alors `Instruction1` sera exécutée sinon `Instruction2` sera exécutée
- ▶ Opération de nullité : `Variable ?? Instruction ;`
 - ▶ Equivalence
`if (Variable == Null) Instruction ;`

Les boucles

- ▶ Quatre boucles comme en JAVA, C, etc.
 - ▶ For :
 - ▶ while :
 - ▶ do ... while() :
 - ▶ foreach :

Les boucles

- For : même principe de fonctionnement



```
1 void main() {  
2   for(int i=0 ; i<=20; i++){  
3     if(i % 2 == 0)  
4       print('$i est pair');  
5   }  
6 }  
7 }  
8
```

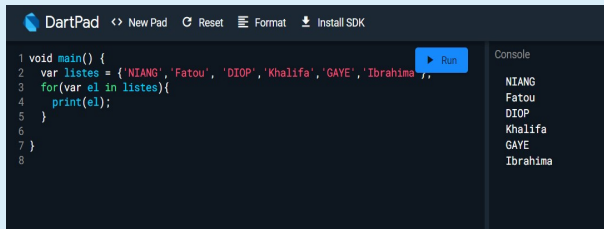
► Run

Console

```
0 est pair  
2 est pair  
4 est pair  
6 est pair  
8 est pair  
10 est pair  
12 est pair  
14 est pair  
16 est pair  
18 est pair  
20 est pair
```

Les boucles

- ▶ foreach : une boucle pour parcourir une collection, un tableau



```
1 void main() {  
2   var listes = {'NIANG', 'Fatou', 'DIOP', 'Khalifa', 'GAYE', 'Ibrahima'},  
3   for(var el in listes){  
4     print(el);  
5   }  
6  
7 }  
8
```

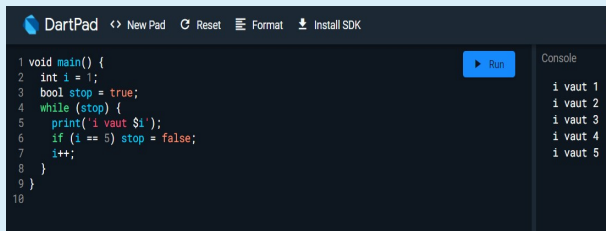
Run

Console

NIANG
Fatou
DIOP
Khalifa
GAYE
Ibrahima

Les boucles

- ▶ while et do...while fonctionnent de la même façon
 - ▶ while est pré-contrôlée
 - ▶ do...while est post-contrôlée
- ▶ Tantque l'expression conditionnelle est vraie, la boucle continue



The screenshot shows the DartPad web IDE. The editor contains the following Dart code:

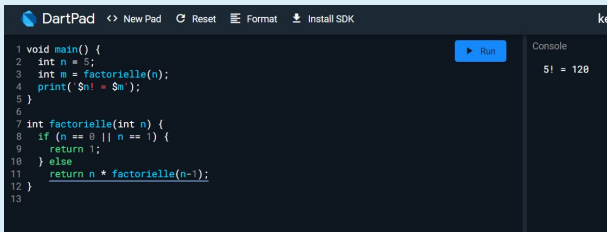
```
1 void main() {  
2   int i = 1;  
3   bool stop = true;  
4   while (stop) {  
5     print('i vaut $i');  
6     if (i == 5) stop = false;  
7     i++;  
8   }  
9 }  
10
```

A blue "Run" button is visible next to the code. To the right, the "Console" panel displays the output of the program:

```
i vaut 1  
i vaut 2  
i vaut 3  
i vaut 4  
i vaut 5
```

Fonctions simples

- ▶ Même principe de fonctionnement qu'en JAVA, C
- ▶ Syntaxe de base : **typeRetour NomFonction(les paramètres)**



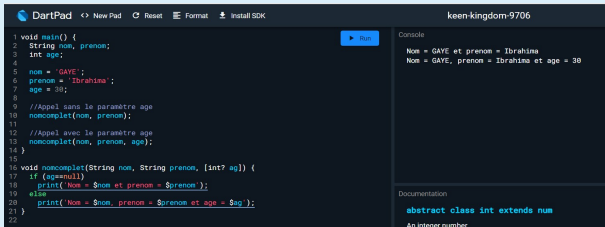
```
1 void main() {  
2   int n = 5;  
3   int m = factorielle(n);  
4   print(' $n! = $m ');  
5 }  
6  
7 int factorielle(int n) {  
8   if (n == 0 || n == 1) {  
9     return 1;  
10  } else  
11    return n * factorielle(n-1);  
12 }  
13
```

Console

5! = 120

Fonctions à paramètres optionnels

- ▶ DART propose des fonctions dont certains paramètres peuvent être optionnels (Notion de polymorphisme)
- ▶ Syntaxe : **typeRetour nomfonction(Par1, ..., [type ? par])**
 - ▶ par peut être utilisé dans l'appel comme il peut ne pas être utilisé



```
1 void main() {  
2   String nom, prenom;  
3   int age;  
4  
5   nom = 'GAYE';  
6   prenom = 'Ibrahima';  
7   age = 38;  
8  
9   //Appel sans le paramètre age  
10  noncomplet(nom, prenom);  
11  
12  //Appel avec le paramètre age  
13  noncomplet(nom, prenom, age);  
14 }  
15  
16 void noncomplet(String nom, String prenom, [int? age]) {  
17   if (age == null) {  
18     print('Nom = $nom et prenom = $prenom');  
19   } else {  
20     print('Nom = $nom, prenom = $prenom et age = $age');  
21   }  
22 }
```

Console

```
Nom = GAYE et prenom = Ibrahima  
Nom = GAYE, prenom = Ibrahima et age = 38
```

Documentation

```
abstract class int extends num  
An integer number.
```

Fonctions fléchées ou fonctions Lambda

- ▶ Similaires aux fonctions classiques
- ▶ différence : la syntaxe est plus courte
- ▶ les accolades seront remplacées par $\Rightarrow$
- ▶ Noter l'absence du mot-clé return dans la fonction lambda avec une valeur à retournée

Fonctions fléchées ou fonctions Lambda



The screenshot shows the DartPad web IDE interface. The top bar includes the DartPad logo, a 'New Pad' button, and icons for 'Reset', 'Format', and 'Install SDK'. The username 'keen-kingdom-9706' is displayed on the right. The main editor area contains the following Dart code:

```
1 void main() {  
2   affichage();  
3   print(affichage());  
4  
5 }  
6  
7 void affichage()=>print('Salut fonction fléchée sans return');  
8  
9 String affichage!()=> 'Salut fonction fléchées avec return';  
10  
11
```

A blue 'Run' button is located to the right of the code. The console on the right side shows the output of the program:

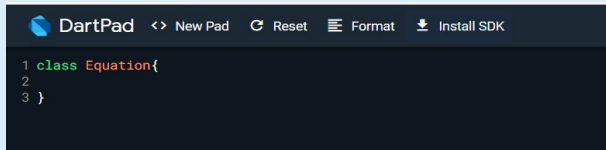
```
Console  
Salut fonction fléchée sans return  
Salut fonction fléchées avec return
```

POO

- ▶ DART est un langage orienté objet
- ▶ Il a implémenté beaucoup de concepts de la POO tels que :
 - ▶ Les classes
 - ▶ L'encapsulation
 - ▶ polymorphisme
 - ▶ L'héritage
 - ▶ La généricité
 - ▶ Interface
 - ▶ etc.

Classe

- ▶ Peut être considéré comme un module dont l'objectif est de pouvoir instancier des objets
- ▶ Le mot-clé **class** est utilisé

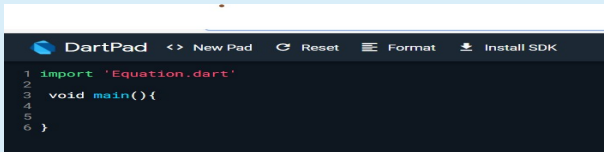


The screenshot shows the DartPad web editor interface. At the top, there is a toolbar with icons for a file, a code editor, and a console, followed by the text 'DartPad'. Below this, there are several buttons: '<> New Pad', 'Reset', 'Format', and 'Install SDK'. The main area of the editor is dark and contains the following code:

```
1 class Equation{  
2  
3 }
```

Classe

- ▶ Elle peut être créée n'importe où ou dans le même fichier DART
- ▶ le fichier DART voulant utiliser la classe va l'importer avec la commande `import Equation.dart`



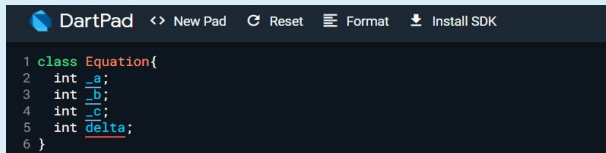
```
1 import 'Equation.dart'
2
3 void main(){
4
5
6 }
```

Attribut

- ▶ Les attributs sont les propriétés des classes
- ▶ Encapsulation : deux protections possibles
 - ▶ private : matérialisé par underscore
 - ▶ public : par une déclaration simple

Attribut

► Exemple



```
DartPad <> New Pad ↺ Reset ≡ Format ⬇ Install SDK

1 class Equation{
2   int _a;
3   int _b;
4   int _c;
5   int delta;
6 }
```

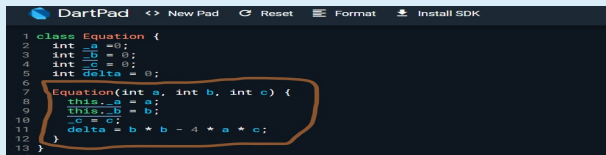
- les attributs `_a`, `_b` et `_c` sont d'encapsulation privée
- l'attribut `delta` est d'encapsulation publique

Constructeurs

- ▶ Une méthode particulière permettant d'instancier des objets de la classe
- ▶ propriétés :
 - ▶ Même nom que la classe
 - ▶ Pas de type de retour
 - ▶ Sans paramètres, on parle de constructeurs par défaut
 - ▶ avec paramètre, on parle de constructeur paramétré
- ▶ Pour lever les ambiguïté, on utilise le mot-clé **this** qui est facultative sinon

Constructeurs

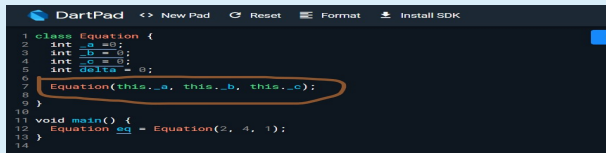
► Exemple



```
1 class Equation {  
2   int _a = 0;  
3   int _b = 0;  
4   int _c = 0;  
5   int delta = 0;  
6  
7   Equation(int a, int b, int c) {  
8     this._a = a;  
9     this._b = b;  
10    _c = c;  
11    delta = b * b - 4 * a * c;  
12  }  
13 }  
14
```

Constructeurs

- ▶ Syntaxe très lourde
- ▶ DART a implémenté une syntaxe beaucoup plus simple

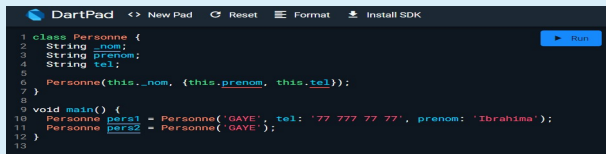


```
1 class Equation {  
2   int _a = 0;  
3   int _b = 0;  
4   int _c = 0;  
5   int delta = 0;  
6  
7   Equation(this._a, this._b, this._c);  
8  
9 }  
10  
11 void main() {  
12   Equation eq = Equation(2, 4, 1);  
13 }  
14
```

The screenshot shows the DartPad interface with a dark theme. The code defines a class `Equation` with four private integer fields: `_a`, `_b`, `_c`, and `delta`. A constructor `Equation(this._a, this._b, this._c);` is highlighted with an orange circle. The `main` function creates an instance of the class: `Equation eq = Equation(2, 4, 1);`.

Constructeurs

- ▶ Parfois, on a besoin d'utiliser plusieurs constructeurs de paramètres différents
- ▶ DART propose des constructeurs où les paramètres peuvent ne pas être présents

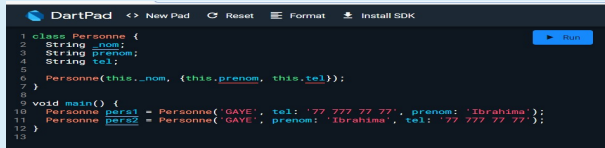


```
1 class Personne {
2   String _nom;
3   String prenom;
4   String tel;
5
6   Personne(this._nom, {this.prenom, this.tel});
7 }
8
9 void main() {
10  Personne pers1 = Personne('GAYE', tel: '77 777 77 77', prenom: 'Ibrahima');
11  Personne pers2 = Personne('GAYE');
12 }
13
```

- ▶ les attributs nom et tel peuvent être utilisés dans n'importe quel ordre

Instantiation

- ▶ Comme dans les diapos précédentes, le mot-clé **new** n'est pas obligatoire
- ▶ S'il y a des paramètres optionnels, on peut les renseigner dans l'ordre que l'on souhaite

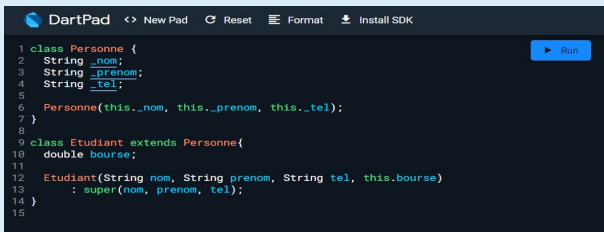


```
1 class Personne {  
2   String _nom;  
3   String prenom;  
4   String tel;  
5  
6   Personne(this._nom, {this.prenom, this.tel});  
7 }  
8  
9 void main() {  
10  Personne pers1 = Personne('GAYE', tel: '77 777 77 77', prenom: 'Ibrahima');  
11  Personne pers2 = Personne('GAYE', prenom: 'Ibrahima', tel: '77 777 77 77');  
12 }  
13
```

- ▶ Ici pers1 et pers2 ont les mêmes informations

Héritage

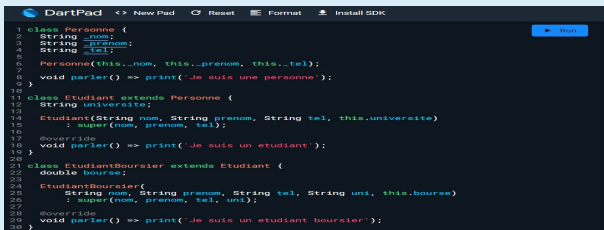
- ▶ Un concept de la POO qui part de la généralisation à la spécification
- ▶ La classe spécifique aura plus de paramètres



```
1 class Personne {
2   String _nom;
3   String _prenom;
4   String _tel;
5
6   Personne(this._nom, this._prenom, this._tel);
7 }
8
9 class Etudiant extends Personne{
10  double bourse;
11
12  Etudiant(String nom, String prenom, String tel, this.bourse)
13    : super(nom, prenom, tel);
14 }
15
```

Héritage

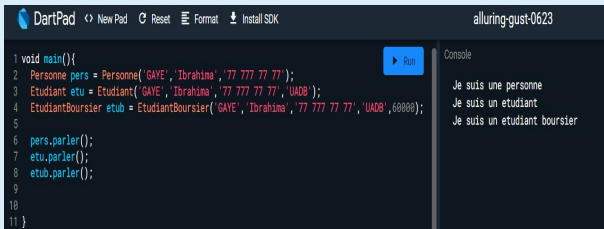
- ▶ La surcharge des méthodes
 - ▶ La même méthode peut être définie plusieurs fois en changeant les paramètres ou/et le contenu



```
1 class Personne {
2   String _nom;
3   String _prenom;
4   String _tel;
5
6   Personne(this._nom, this._prenom, this._tel);
7   void parler() => print('Je suis une personne');
8 }
9
10
11 class Etudiant extends Personne {
12   String universite;
13
14   Etudiant(String nom, String prenom, String tel, this.universite)
15     : super(nom, prenom, tel);
16
17   @override
18   void parler() => print('Je suis un etudiant');
19 }
20
21 class EtudiantBoursier extends Etudiant {
22   double bourse;
23
24   EtudiantBoursier(
25     String nom, String prenom, String tel, String uni, this.bourse)
26     : super(nom, prenom, tel, uni);
27
28   @override
29   void parler() => print('Je suis un etudiant boursier');
30 }
```

Héritage

- ▶ Dans l'exemple, nous trois classes :
 - ▶ Personne
 - ▶ Etudiant qui herite de Personne
 - ▶ EtudiantBoursier qui hérite de Etudiant



```
1 void main(){
2   Personne pers = Personne('GAYE', 'Ibrahima', '77 777 77 77');
3   Etudiant etu = Etudiant('GAYE', 'Ibrahima', '77 777 77 77', 'UADB');
4   EtudiantBoursier etub = EtudiantBoursier('GAYE', 'Ibrahima', '77 777 77 77', 'UADB', 60000);
5
6   pers.parler();
7   etu.parler();
8   etub.parler();
9
10
11 }
```

alluring-gust-0623

Console

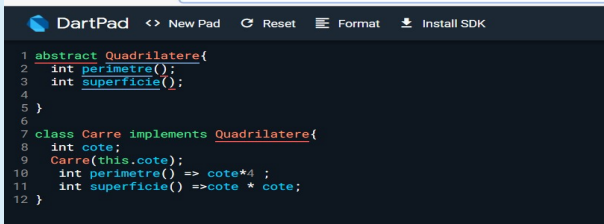
```
Je suis une personne
Je suis un etudiant
Je suis un etudiant boursier
```

Interface

- ▶ En programmation, on peut connaître l'existence d'un comportement d'un objet sans avoir les détails de l'implémentation
 - ▶ Dans une Classe Quadrilatère, la **superficie** existe
 - ▶ Mais les détails de l'implémentation peut dépendre de sa nature
 - ▶ Le comportement superficie sera définie comme une **méthode abstraite**
- ▶ Une Interface est classe où toutes les méthodes sont **abstraites**
- ▶ Instancier une interface est sans intérêt
 - ▶ On n'a pas besoin d'un constructeur
- ▶ Elle sera implémentée par une classe

Interface

► Exemple



```
DartPad <> New Pad ↺ Reset ≡ Format ⬇ Install SDK

1 abstract Quadrilatre{
2   int perimetre();
3   int superficie();
4 }
5
6
7 class Carre implements Quadrilatre{
8   int cote;
9   Carre(this.cote);
10  int perimetre() => cote*4 ;
11  int superficie() =>cote * cote;
12 }
```